DOCUMENTO MAESTRO

SmartOrder AI

Sistema Inteligente de Pedidos Sugeridos para la Gestión Comercial Avícola
Vitali Alimentos

Memoria técnica académica
origin/main · 29 de octubre de 2026
SHA 5129401fd8e346f088ba377aff039d44eb03a1d3

Índice

1. Resumen ejecutivo

2. Introducción

3. Descripción del problema

4. Justificación

5. Objetivos

6. Alcance

7. Evolución del proyecto

8. Arquitectura

9. Stack tecnológico

10. Estructura del repositorio

11. Modelo de datos

12. Usuarios y roles

13. Gestión de ventas

14. Inventario

15. Motor de recomendaciones

16. Inteligencia Artificial

17. Pedido sugerido

18. Planificación

19. Flujo comercial

20. Producción

21. Integración Odoo

22. Dashboards

23. Reportes

24. Streamlit

25. Django

26. Seguridad

27. Pruebas y resultados técnicos comprobados

28. Resultados de negocio esperados

29. Limitaciones

30. Deuda técnica

31. Estado actual vs. visión objetivo

32. Próximos pasos

33. Conclusiones

1. Resumen ejecutivo

SmartOrder AI es un prototipo de apoyo a la gestión comercial avícola. En el HEAD auditado combina Django y Streamlit, importa ventas e inventario, compara métodos de pronóstico y presenta recomendaciones que una persona debe revisar. Una venta confirmada no autoriza automáticamente producción. La operación Odoo incluida es un laboratorio local con datos sintéticos, no una implantación empresarial.

2. Introducción

Esta memoria describe el estado de origin/main al 29 de octubre de 2026, commit 5129401fd8e346f088ba377aff039d44eb03a1d3. La fuente principal es el código, migrations, pruebas, scripts y README. Distingue funciones implementadas, simulación y aspectos no validados en infraestructura externa.

3. Descripción del problema

El caso parte de pedidos apoyados en hojas de cálculo y criterio individual, con riesgo de estimar por encima o por debajo de la demanda, reaccionar tarde y perder trazabilidad. La fragmentación entre ventas, inventario y producción dificulta anticipar disponibilidad y compromisos. Son problemas del contexto del proyecto; no se midieron aquí tasas reales de quiebre o merma en Vitali.

4. Justificación

Centralizar ventas, disponibilidad, reglas y aprobaciones ayuda a que vendedor, administración y operación trabajen con datos comparables. La recomendación reduce cálculos manuales repetidos, mientras la revisión humana conserva responsabilidad comercial. Los beneficios económicos requieren un piloto con datos observados y grupo/periodo de comparación.

5. Objetivos

Objetivo general: apoyar la gestión comercial avícola con datos históricos, analítica, pronóstico, inventario y reglas de negocio, dentro de un flujo supervisado por personas. Objetivos específicos: centralizar cargas, reducir errores de transcripción, apoyar planificación, hacer trazables ajustes y aprobaciones, evaluar métodos predictivos por producto e integrar gradualmente un ERP.

6. Alcance

Incluye dos interfaces (Django y Streamlit), usuarios y permisos, carga Excel, recomendaciones, inventario, pedidos/ventas, autorización de producción, reportes y laboratorio Odoo. No incluye servicio productivo confirmado, integración fiscal, conectividad Odoo Cloud ni resultados económicos demostrados. Streamlit Cloud usa almacenamiento efímero y no es apropiado para datos comerciales reales.

7. Evolución del proyecto

Línea de tiempo basada en fechas del historial Git: 2026-09-25, `a74f37b` inicia el sistema local de pedidos sugeridos; `c555cb7` incorpora dashboards y pedidos revisados; `515a3d6` construye el prototipo avanzado; `ea6c308` activa una demo aislada en Streamlit Cloud; `411104b` evita inicializaciones públicas concurrentes; `d45ff0b` documenta esa demo. PR #1 se integra en `c548848` y PR #2 en `a19cbee`. El 2026-10-04, `1830457` reemplaza la demo pública aislada por Streamlit compartido y `9a533bc` reduce el mínimo de contraseña a ocho caracteres. El 2026-10-29, el HEAD `5129401` amplía el piloto comercial y la planificación de pedidos.

El historial confirma etapas por commits, no por el nombre del producto solamente. `a74f37b` inicia sistema local; `c555cb7` añade paneles y pedidos revisados; `515a3d6` crea SmartOrder avanzado; `ea6c308`/`411104b` introducen demo pública y concurrencia; `d45ff0b` documenta su publicación. Los merges `c548848` y `a19cbee` identifican la integración de PR #1 y #2. En septiembre se registra el cambio a Django y Streamlit; para el 4 de octubre README resume laboratorio Odoo y 83 tests; el

commit `1830457` sustituye la demo aislada por interfaz Streamlit compartida; `9a533bc` reduce el mínimo a ocho; `5129401` amplia piloto y planificación.

La diferencia de arquitectura es material: el `app.py` monolítico de la versión anterior se reduce a wrapper, se eliminan la autenticación/almacenamiento propios y el seed demo anterior, y se añaden 14 migraciones Django, dominio operacional, bridge Streamlit, configuraciones y scripts de laboratorio. La historia 83→134 pruebas representa expansión de cobertura, no una comparación de calidad estadística.

8. Arquitectura

Django mantiene autenticación, ORM, permisos y servicios. La UI Django usa templates; Streamlit usa `streamlit_app.py` y un bridge interno hacia el dominio compartido. Las importaciones pasan por validadores; los servicios calculan recomendaciones y reglas comerciales; los registros se guardan en base de datos y archivos locales. Odoo se integra mediante conector y addon del laboratorio.

El request HTTP entra por URLconf de `operations/urls.py`. Vistas aplican autenticación, rol y estado; formatean contextos y delegan cambios a servicios. Los servicios validan el estado actual de fuente, unidad, asignaciones, inventario y solicitud bajo transacciones Django. Modelos materializan procedencia, fechas, confirmaciones y referencias. `streamlit_app.py` no importa credenciales ni mantiene otro repositorio de negocio; `streamlit_bridge.py` ejecuta la misma capa de dominio.

En despliegue Streamlit cada acción puede iniciar Django en proceso si la base es SQLite, conforme al entrypoint y bridge; comparten la ruta de DB/media local. Eso favorece reglas consistentes entre UI, pero significa que Cloud requiere storage durable y configuración estable. El diagrama simplifica flujos, no representa cada tarea ni cada relación del ORM.

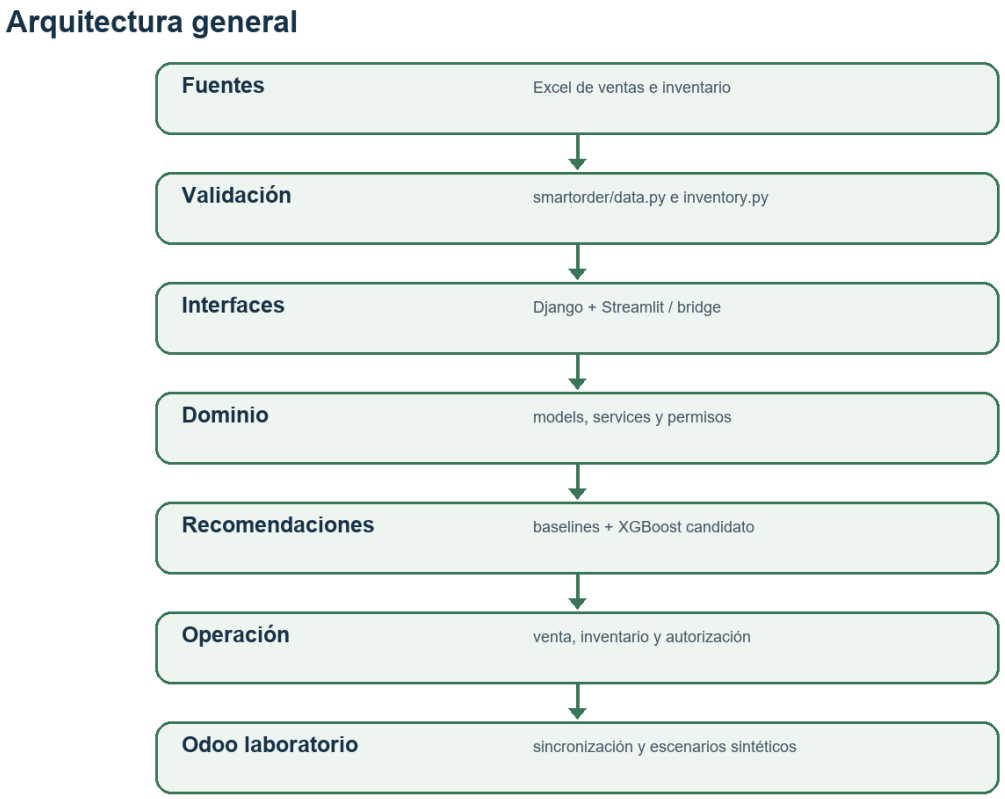


Figura 1. Arquitectura general

9. Stack tecnológico

Python implementa lógica; Django maneja web, usuarios y ORM; Streamlit entrega UI interactiva; pandas/NumPy procesan series; openpyxl lee y escribe Excel; scikit-learn aporta métricas; XGBoost es candidato de pronóstico. SQLite es la configuración local; PostgreSQL está contemplado por configuración/dependencia opcional. Bootstrap y Chart.js apoyan Django; Telegram es opcional. Odoo Community y GitHub aparecen como laboratorio e historial/código.

El entorno auditado con Python 3.12 resolvió Django 5.2.18, Streamlit 1.66.1, pandas 2.3.3, NumPy 2.5.4, openpyxl 3.1.5, XGBoost 3.4.1, scikit-learn 1.9.1 y psycopg 3.3.6. Los manifiestos expresan rangos, no lockfile; versiones instaladas describen esta verificación, no un pin permanente.

PostgreSQL aparece como opción por variable `SMARTORDER_DATABASE_URL` y requirements-postgres, pero la prueba ejecutada usó SQLite. La integración Odoo usa su API/modelos vía `operations/odoo.py` y addon propio; los documentos mantienen reservas, fabricación, logística y contabilidad en Odoo, no replican esas funciones como ERP en SmartOrder.

Panel administrativo de Django con indicadores de ventas y acciones pendientes. Usuario: administrador. Los valores provienen del dataset sintético del laboratorio; permite revisar ventas filtradas y trabajo operativo.

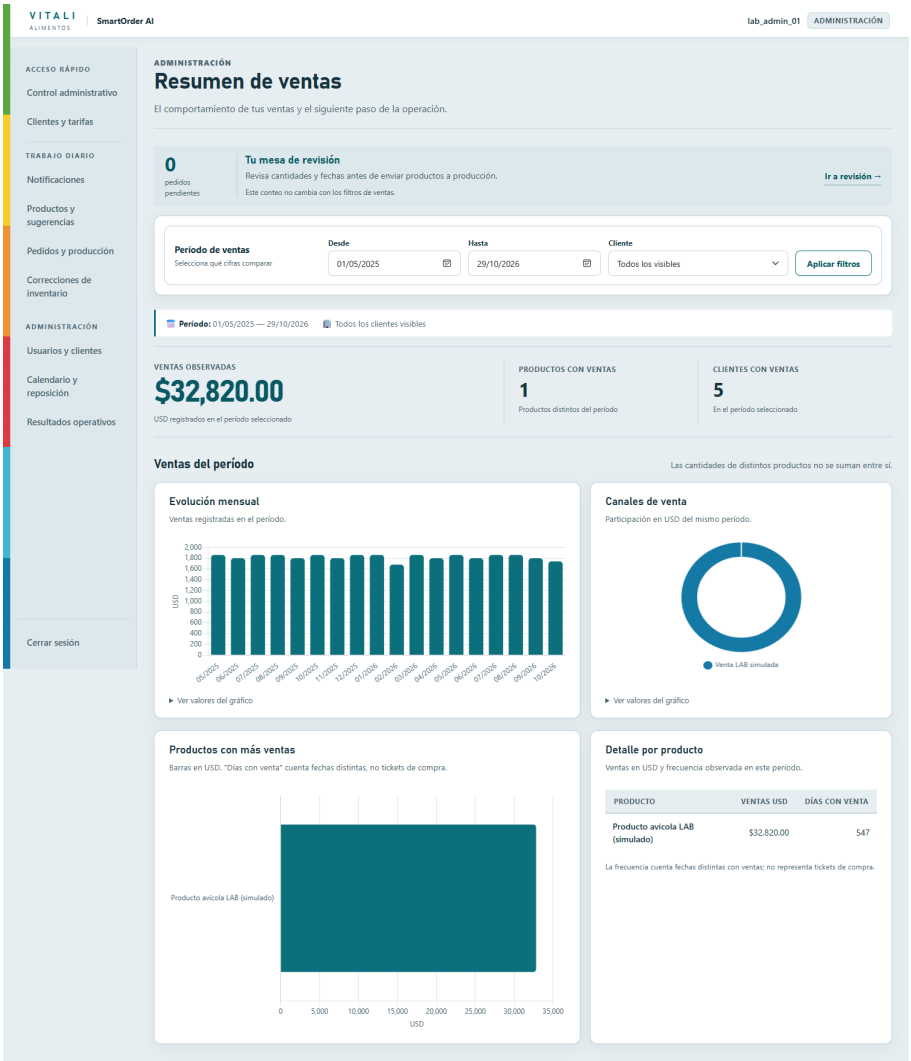


Figura 2. Panel administrativo de Django con indicadores de ventas y acciones pendientes

10. Estructura del repositorio

`operations/`: modelos, servicios, vistas, permisos, templates, estáticos, migraciones, integración y pruebas. `smartorder/`: datos, inventario, recomendaciones, exportación y avisos. `vitali_web/`: settings, URLconf y WSGI. `odoo_addons/vitali_lab/`: addon del laboratorio. `scripts/`: setup y escenarios de respaldo, Odoo y operación. `tests/`: pruebas de dominio. `manage.py`, `streamlit_app.py` y `app.py` son entradas relevantes; `requirements*.txt` declara dependencias.

`operations/models.py` y migrations mantienen el estado persistente. `services.py` transforma fuentes y solicitudes; `commercial.py` aplica precio y métricas del piloto; `odoo.py` administra referencia, revisión, sincronización y snapshots. `operations/templates/operations/` contiene pantallas Django; CSS/JS y librerías vendor son recursos estáticos. `operations/test_*.py` separa grupos de pruebas UI, integración Odoo, piloto y bandeja de planificación; `operations/tests.py` cubre reglas centrales.

Los scripts realizan tareas explícitas de preparación, escenarios, respaldo, reconciliación, exportación e importación de legado. El iniciador `iniciar_laboratorio.cmd` selecciona Django (8001), Streamlit (8502) u Odoo (8079); `iniciar_smartorder.cmd` y `iniciar_streamlit.cmd` arrancan modo local normal y migran DB antes de atender usuarios.

11. Modelo de datos

Las entidades centrales observadas incluyen SalesDataset/SalesLine (fuentes y filas), Assignment (cartera), InventoryBatch/InventoryItem/DatedFlow (cortes, existencia y movimientos), OrderRequest/OrderLine (solicitud y detalle), PriceTariff (precio/regla), SourceSync y mapeos Odoo, correcciones de stock, notificaciones/eventos y registros de usuario/rol. Las relaciones y restricciones concretas se definen en `operations/models.py` y 14 migraciones. El diagrama incluido es simplificado; no sustituye al ERD completo.

Entidades de fuente: `SalesDataset` conserva hash, fichero, procedencia (unknown/real/simulated), semántica de fecha, cobertura, autor y si se confirmó snapshot completo; `SalesLine` conserva unidad, fecha de pedido/entrega, referencia de origen, cantidad, precio y monto. `Assignment` asocia una cartera de cliente a usuario. Restricciones evitan más de un dataset activo y cantidades/precios negativos.

Inventario: `InventoryBatch` identifica import y unidad confirmada; `InventoryItem` guarda cliente-producto-unidad-valor-fecha; `DatedFlow` modela entradas o compromisos con fecha/estado de conciliación y posible OrderLine asociada. `ClientStockCount`, `ClientStockMovement`, `InventoryAdjustment`, `LocationStockSnapshot`, `LocationIncoming` y `RestockTarget` conservan otras vistas/ajustes de stock y reposición.

Comercial y control: `OrderRequest` y `OrderLine` separan cabecera de línea, snapshot de fuente y estados/revisión; `OrderCoverage`, `SalesCoverage` y `SalesOperationalReconciliation` controlan cobertura y duplicidad; `PriceTariff` aplica cliente-producto-unidad; `CommercialEvent` representa eventos/fechas; `StockCorrectionRequest`, `NotificationLog`, `UserNotification` y `AccessEvent` registran solicitudes, alertas y auditoría.

Integración: `SourceSync` guarda ejecución, filas aceptadas/rechazadas, huella y error; `OdooCustomerMapping`/`OdooProductMapping` mapean identificadores; `OdooOrderLink` guarda referencia UUID, revisión, estado, snapshot y último error. El ERD simplificado no enumera todos los modelos de identidad Django ni cada FK.

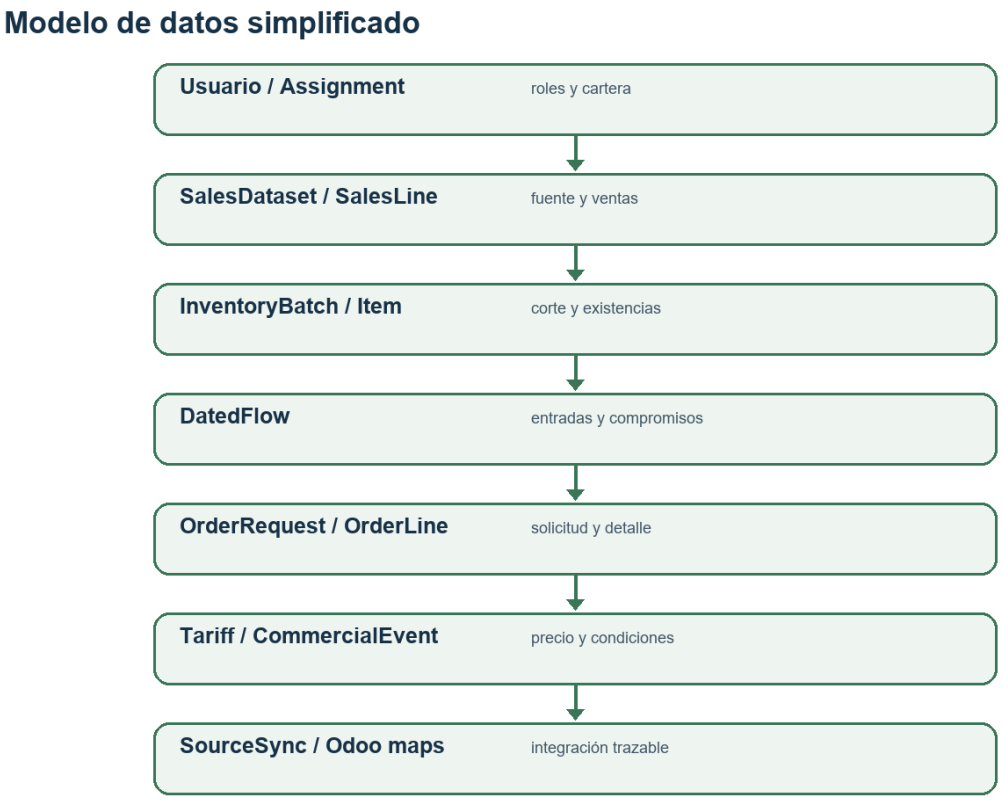


Figura 3. Modelo de datos simplificado

12. Usuarios y roles

Administrador: carga y activa fuentes, asigna cartera, administra usuarios, revisa indicadores, inventario, recomendaciones, solicitudes, tarifas, producción y configuración Odoo. Vendedor: consulta cartera propia, disponibilidad y sugerencias permitidas, registra y sigue ventas, y solicita correcciones. No accede a fuentes históricas completas, finanzas o credenciales administrativas. No hay un rol productivo separado documentado como usuario final.

Pantalla de acceso Django antes de autenticar. Es el punto de entrada compartido del frontend web; no contiene datos de cuenta.

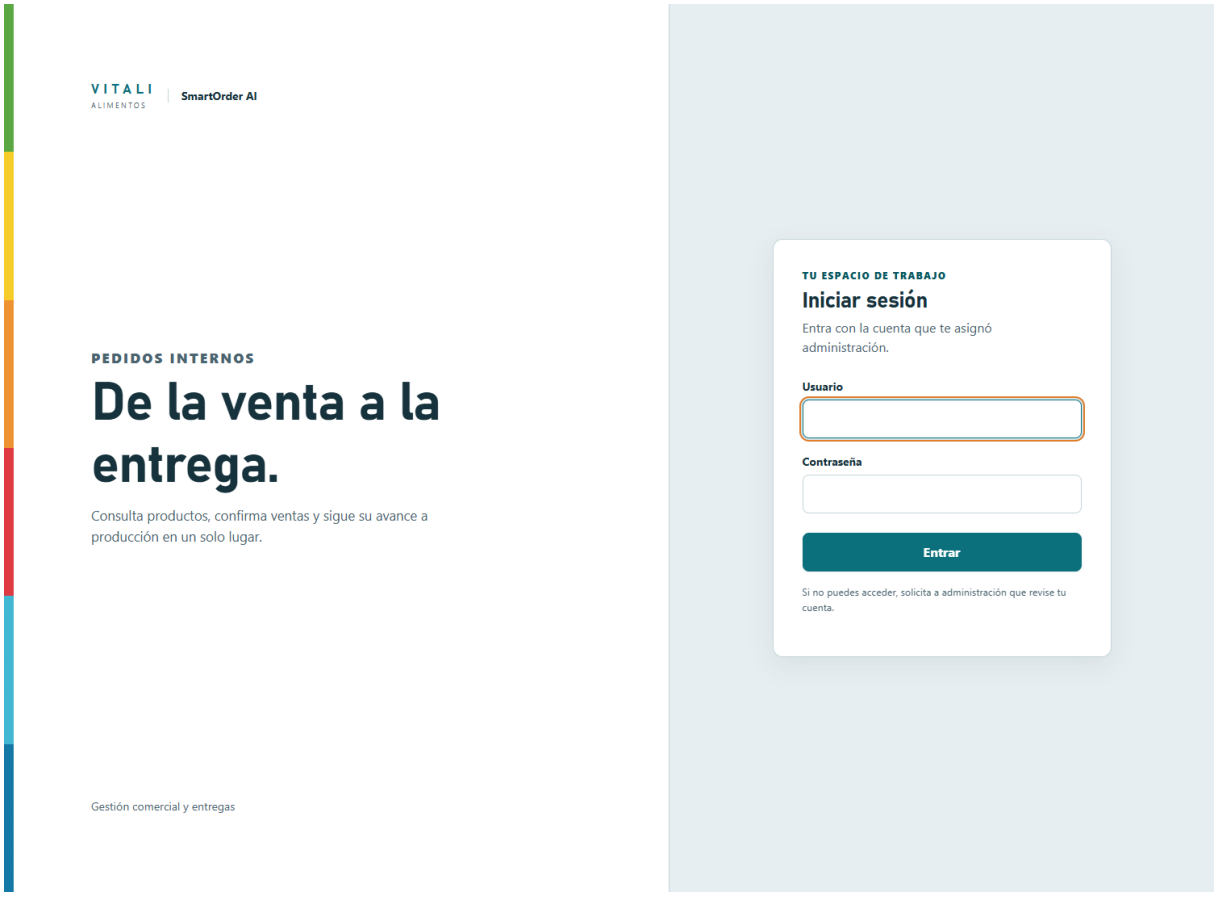


Figura 4. Pantalla de acceso Django antes de autenticar

13. Gestión de ventas

La importación recibe libros Excel y valida estructura, fechas, valores, unidades y consistencia. Se versionan datasets y se conserva procedencia; activar un archivo establece la fuente activa y evita mezclar silenciosamente cargas parciales. Hay herramientas de importación/migración de legado. La venta confirmada registra cantidades, precio, descuento, condiciones y fecha comprometida según las validaciones de servicios.

La ingestión es de snapshot completo. `import_sales` valida hojas/cabeceras equivalentes, fechas, cantidades, precio, monto, unidades y referencias; conserva la fila y la fuente. Activar una nueva versión no concatena silenciosamente una semana parcial: README requiere que cada Excel contenga todo el histórico a conservar. Metadatos guardan digest, archivo, autor, rango, fila, verified y full_snapshot_confirmed.

El modelo distingue fecha de venta/pedido de fecha de entrega realizada. Antes de usar ventas operativas en históricos puede requerirse revisión de solapamiento/reconciliación para no contar dos veces una operación que después aparece en Excel. Filas inválidas deben reportarse, no convertirse en demanda implícita. Las pruebas cubren reemplazo semanal y revalidación de pedidos contra fuente nueva.

14. Inventario

El módulo `smartorder/inventory.py` acepta existencia por cliente/producto/unidad y fecha de corte, además de movimientos fechados. Entradas, compromisos y ajustes actualizan la disponibilidad proyectada. Las unidades se comparan por separado y deben estar confirmadas. Una corrección requiere trazabilidad; el código y las pruebas verifican reconciliación y origen de lecturas.

La plantilla de contingencia registra cliente, producto, existencia disponible, fecha de corte y unidad; `InventoryBatch` guarda digest, archivo, fuente, confirmación de match de unidad y activo. La existencia de snapshot no equivale a reserva física: los

movimientos fechados se ingresan aparte y Odoo mantiene reserva real dentro del laboratorio.

La planificación proyecta stock al día relevante sumando existencia confirmada y entradas previas y restando compromisos anteriores; unidad y fecha definen si se puede comparar. Los límites de serie, inventario vencido, unidad desconocida o contradicción de fuente bloquean automatismo. Un conteo/corrección se mantiene trazable al origen y no reescribe la fila importada.

Centro de fuentes muestra archivos, procedencia y vigencia. Administrador; datos Excel sintéticos. Permite verificar qué versión alimenta indicadores y recomendaciones.

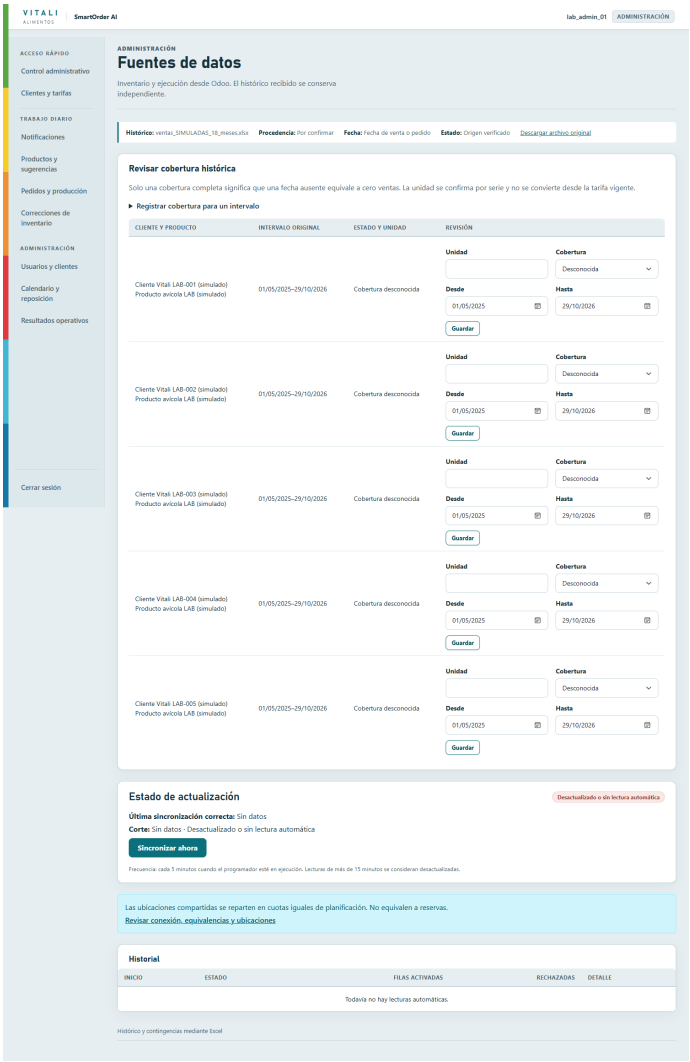


Figura 5. Centro de fuentes muestra archivos, procedencia y vigencia

15. Motor de recomendaciones

`smartorder/recommendations.py` construye referencias de demanda por cliente-producto, compara ventanas y métodos de promedio con XGBoost, mide error por producto y evita recomendar automáticamente cuando falta evidencia, inventario vigente o unidad inequívoca. El motor entrega explicación y puede caer a modo manual; no ejecuta transacciones de venta o fabricación.

`METHODS` define cinco candidatos: últimos 7 días, promedio de 28 días, promedio de 56 días, mismos siete días del año anterior y XGBoost. El método se evalúa por serie cliente-producto y fecha inicial de entrega; no hay obligación de usar el modelo complejo si un baseline ofrece mejor evidencia. Para las comparaciones estacionales se requiere historia de la misma época y fechas alineadas.

El flujo de ventas del piloto construye 16 orígenes semanales comparables: ocho ventanas de selección y ocho posteriores de auditoría independiente. El periodo/año anterior puede ser feature/método candidato según disponibilidad, pero la documentación no lo declara ganador por defecto. La serie necesita cobertura suficiente: la evaluación separa selección contra baseline de 28 días y auditoría fuera de la selección; sin auditoría/calidad, entrega cae en modo manual.

El `XGBRegressor` usa parámetros deterministas (incluye `random_state=73`, `n_jobs=1`, depth 3, learning rate 0.05); su entrenamiento usa features temporales/rezagos calculados en `_features`, escala objetivo por producto y requiere al menos ocho ejemplos. No se persiste un bundle de modelo: se calcula en la evaluación a partir del dataset activo y cachés en proceso. No es evidencia de causación, ni de generalización a productos/sucursales no representados.



Figura 6. Flujo de recomendaciones IA

Carga/consulta de inventario sintético con fecha y unidad. Administrador; ayuda a revisar disponibilidad para compromisos.

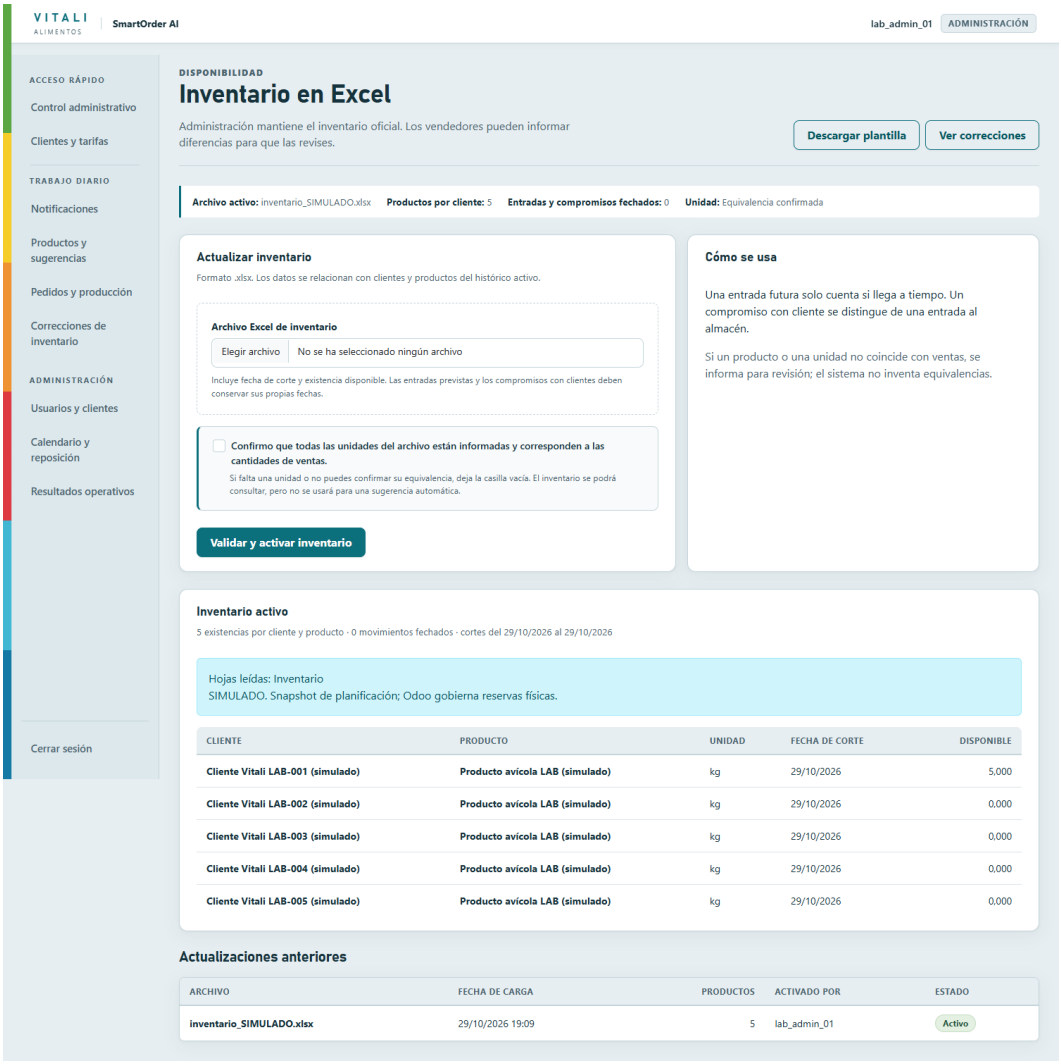


Figura 7. Carga/consulta de inventario sintético con fecha y unidad

16. Inteligencia Artificial

XGBoost es un regresor de árboles y solo un candidato entre baselines, no una garantía de que siempre gane. La preparación usa historia de ventas transformada en variables temporales/de calendario y rezagos según funciones de recomendaciones. La salida se compara contra valores observados en cortes cronológicos. No se encontró evidencia de modelo serializado/persistido como artefacto; el cálculo depende de datos y versión de librerías.

La selección no reutiliza la misma semana como entrenamiento y auditoría: la parte posterior se mantiene independiente. `SELECTION_WINDOWS` y el reporte contabilizan periodos y series elegibles; la cobertura se muestra junto a los resultados. Una serie puede no estar calificada aun cuando haya un número de forecast, porque control de calidad, frescura, unidad confirmada e inventario vigente son precondiciones operativas distintas del cálculo estadístico.

WAPE relaciona suma de errores absolutos con suma de demanda observada; resulta interpretable como error relativo agregado de cada producto, aunque con denominador cero puede no estar disponible. El error absoluto conserva unidad y mide escala; el exceso de forecast mide sobreestimación contra el baseline para limitar sesgo de sobreproducción. README/code no identifican RMSE o MAPE como criterio actual de selección; no se reportan como métricas implementadas. Feature importance explica contribuciones internas del modelo, no causalidad.

Página Django de productos y sugerencias. Administrador; usa ventas sintéticas, backtest e inventario; permite revisar elegibilidad/manualidad.

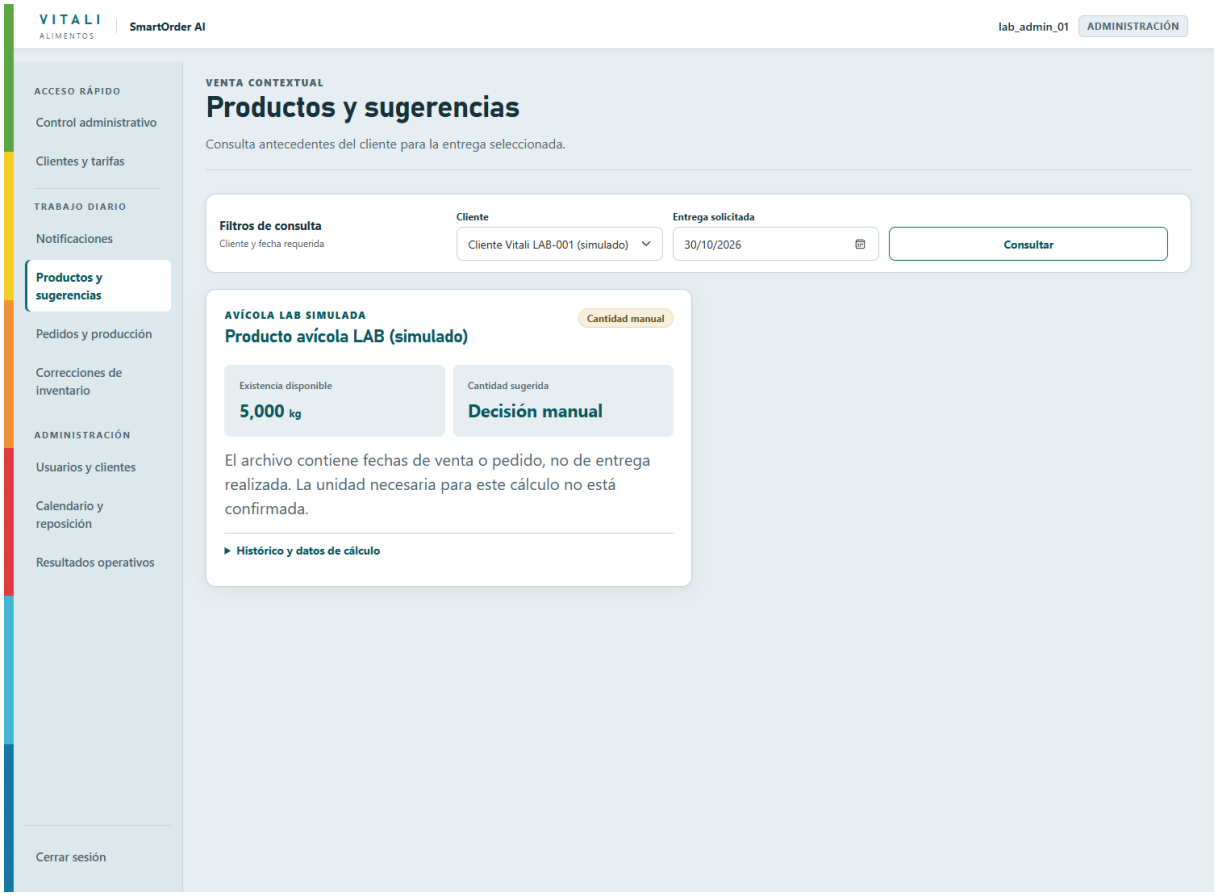


Figura 8. Página Django de productos y sugerencias

17. Pedido sugerido

Fórmula conceptual original: demanda prevista + inventario objetivo – disponible – pedidos pendientes. Implementación actual: cantidad orientativa basada en el máximo entre demanda proyectada y compromisos del periodo, menos inventario proyectado a entrega (existencia actual + entradas previas – compromisos previos), con mínimo cero. El ajuste final está sujeto a unidad, evidencia y reglas comerciales. No confundir la formulación conceptual con la regla actual.

Fórmula conceptual vs. lógica implementada. La formulación inicial era `demanda prevista + inventario objetivo – inventario disponible – pedidos pendientes`; sirve como marco intuitivo para reponer stock. El motor actual separa demanda esperada y flujo comercial fechado. Calcula el máximo entre demanda proyectada en el horizonte y compromisos de cliente, y resta inventario proyectado al compromiso (stock vigente más entradas previas menos compromisos anteriores), con límite inferior en cero. No se fuerza equivalencia algebraica porque las definiciones y periodos del inventario objetivo/pedidos pendientes se sustituyeron por disponibilidad temporal comprobable.

La salida automática requiere fuente aceptada y reciente, cobertura/evaluación independiente suficiente, serie que pasa umbrales, inventario vigente y unidad inequívoca. La fecha seleccionada debe estar en la ventana que acepta el flujo (venta hasta 365 días; sugerencia con horizonte cercano), las características de entrega alimentan la semana objetivo y la API impide mezclar unidades. Si una condición falla, vendedor puede introducir cantidad manual con justificación y la interfaz la marca como manual; no se finge confianza numérica.

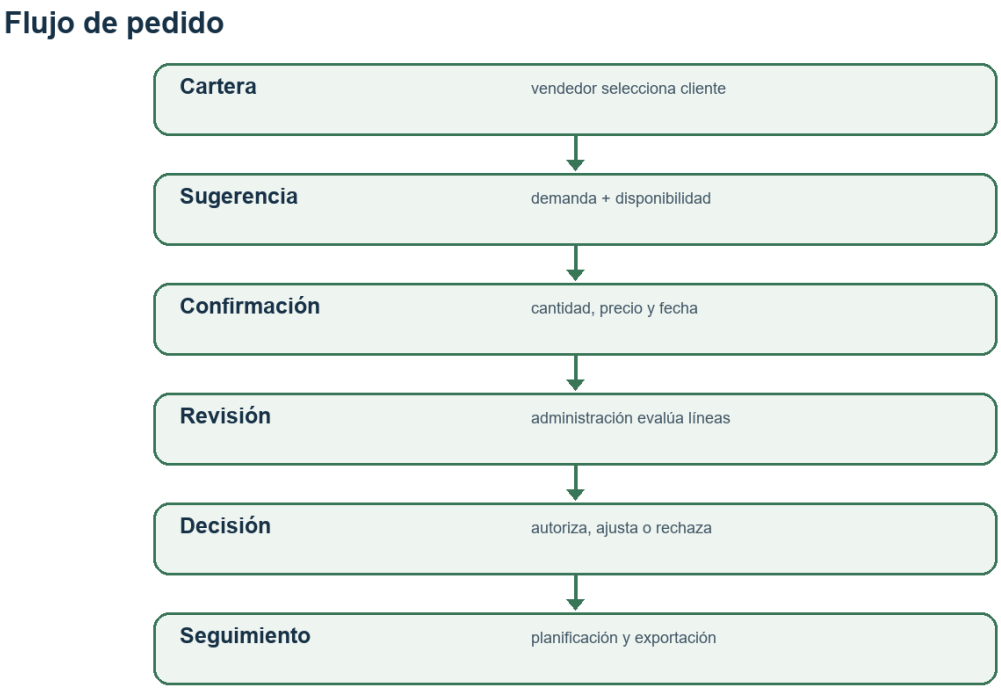


Figura 9. Flujo de pedido

Formulario real de nueva venta del vendedor: cliente, entrega, producto, cantidad, tarifa y condiciones. Usa cartera/producto sintéticos; permite revisar una propuesta antes de confirmarla.

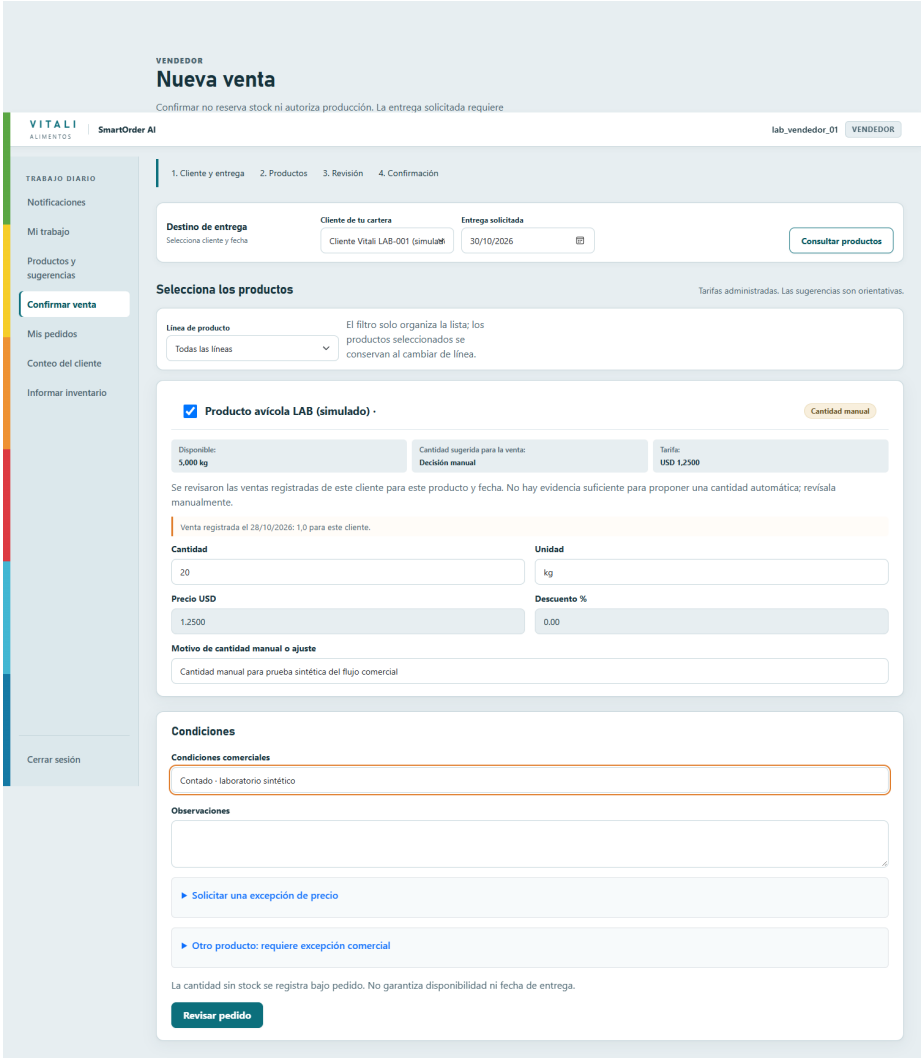


Figura 10. Formulario real de nueva venta del vendedor: cliente, entrega, producto, cantidad, tarifa y condiciones

18. Planificación

El sistema presenta una fecha de entrega dentro del horizonte permitido, compromisos y disponibilidad, y una bandeja/planificación futura. El último commit incorpora más flujo de planificación y revalidación; la vista debe interpretarse como apoyo operativo a solicitudes y fabricación, no como optimizador de planta ni MRP integral.

La planificación diferencia calendario de entrada esperada, compromiso de cliente, solicitud y autorización productiva. Los flujos fechados tienen semántica requested/committed/expected incoming y reconciliación pending/unique/duplicate/ambiguous. Una entrada planificada no autoriza fabricar; una fecha solicitada no es automáticamente fecha comprometida; una confirmación de unidad o movimiento afecta la proyección. Views/services revalidan frente a nueva fuente y permiten bandeja de planificación con estados/acciones.

El último commit incluye migraciones `0008_planning_inbox`, `0009_location_planning` y migraciones de flujos/reconciliación que agregan ubicación, entradas esperadas y control de duplicados. Esto mejora trazabilidad temporal pero no sustituye una herramienta de planificación de capacidad finita, secuenciación de líneas ni MRP corporativo.

Calendario y reposición con flujos fechados del laboratorio. Administrador; ayuda a coordinar compromisos y entradas proyectadas.

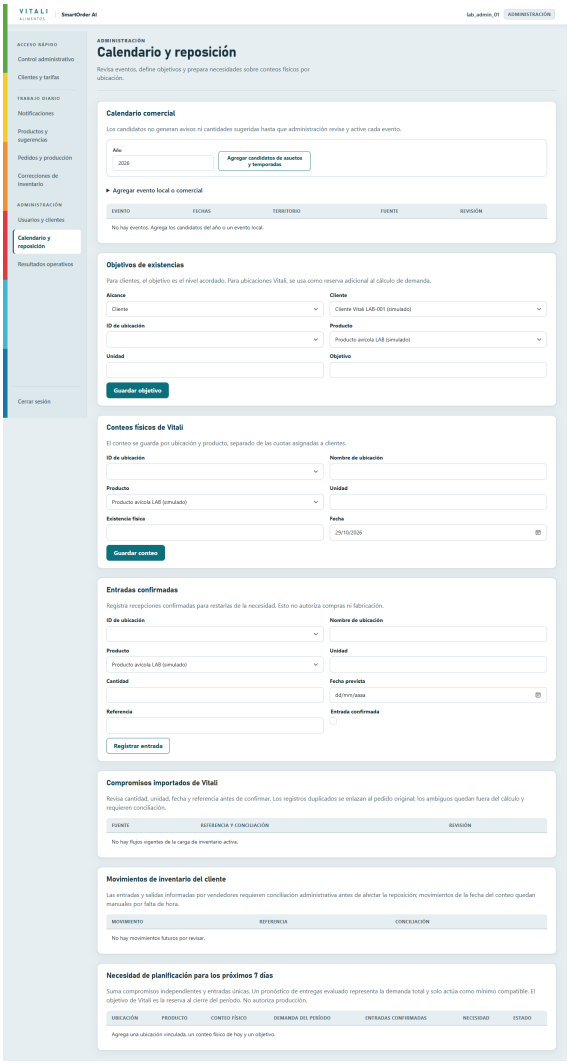


Figura 11. Calendario y reposición con flujos fechados del laboratorio

19. Flujo comercial

Vendedor selecciona cliente de su cartera, producto, cantidad, precio, descuento, condiciones y fecha comprometida; explica una excepción cuando las reglas la solicitan y confirma la venta. Administración consulta y mantiene trazabilidad. La recomendación se puede revisar y ajustar; la venta confirmada no es equivalente a una orden de producción.

Al confirmar una venta, el vendedor elige cliente de cartera, fecha, producto(s), cantidad, unidad, precio vigente, descuento y condiciones, puede documentar excepción comercial/notas y mantiene un token de envío para idempotencia. Se revalidan tarifas y snapshot antes de aceptar; cambios incompatibles requieren revisión. Cancelación/modificación tienen estados y motivo; las acciones no deben interpretarse como venta facturada o entrega completada.

`OrderRequest` guarda vendedor, cliente, fuente original y dataset revalidado, submission UUID/hash, notas/excepciones y fechas de revisión. `OrderLine` mantiene SKU/producto/unidad/cantidad/importe y relación con autorizaciones/flujo Odoo. Las protecciones de rol impiden al vendedor cargar datos, administrar usuarios, aprobar producción o exportar lotes.

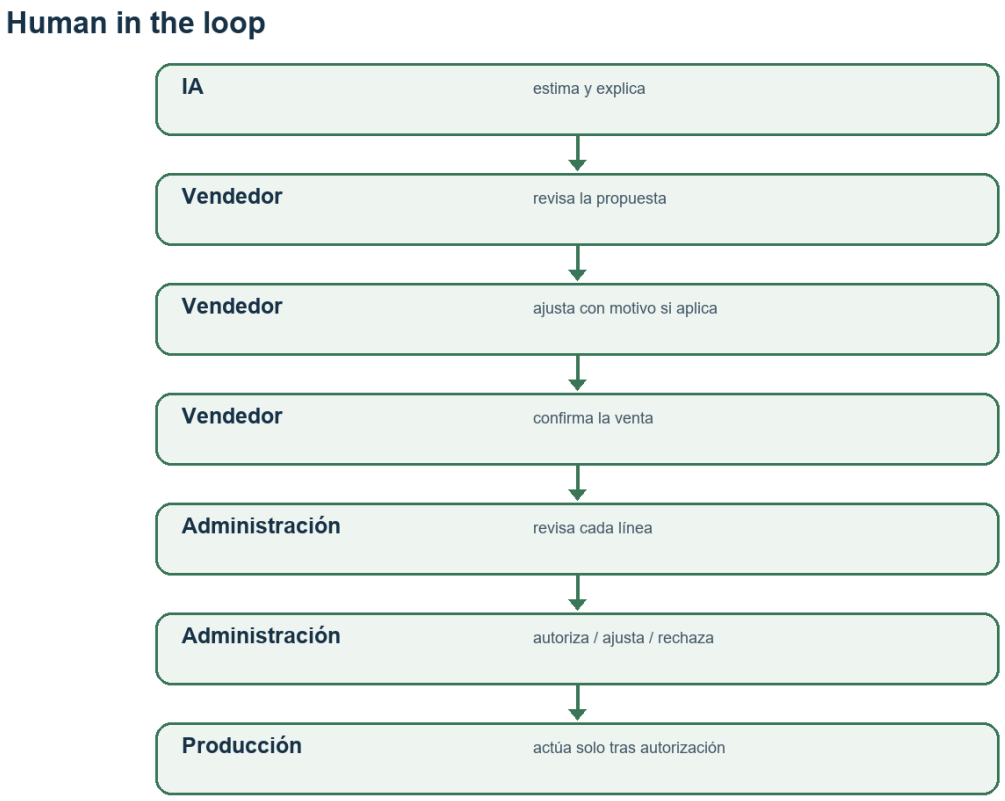


Figura 12. Human in the loop

Detalle real de la venta sintética confirmada por el vendedor. Presenta cliente, entrega solicitada, precio, cantidad y estado de producción pendiente; permite dar seguimiento sin confundir confirmación comercial con autorización productiva.

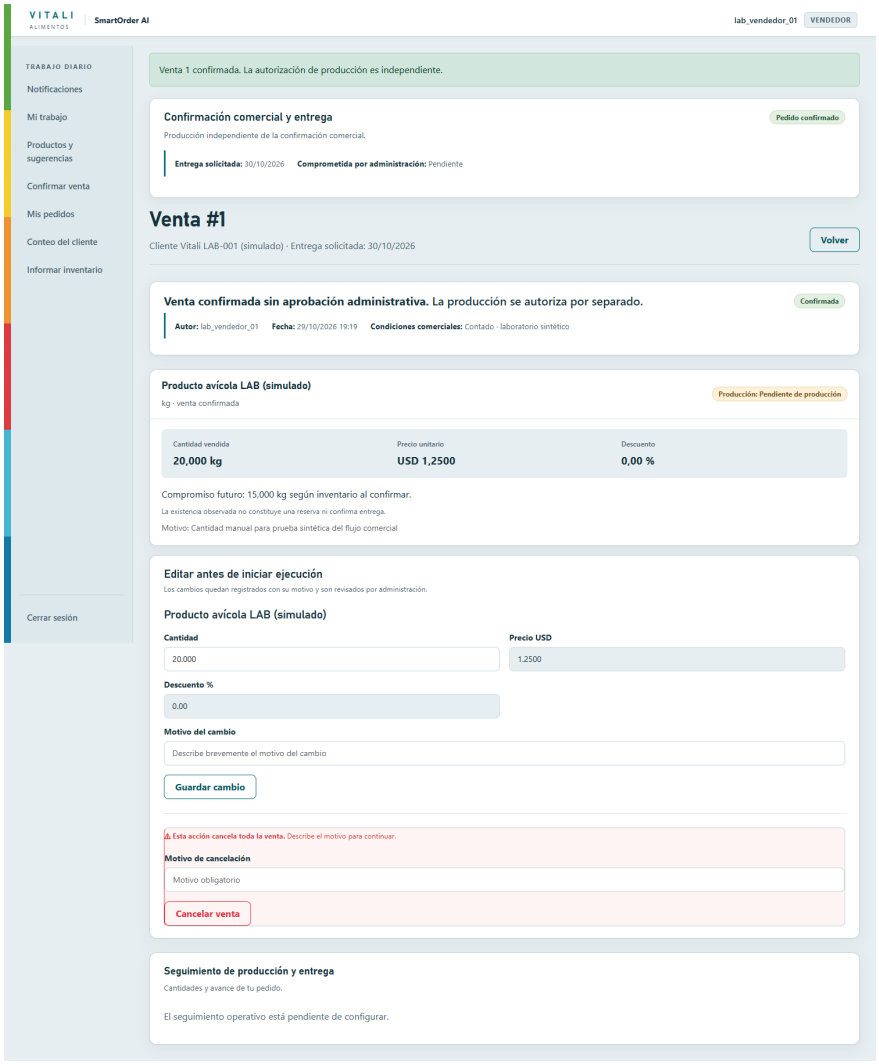


Figura 13. Detalle real de la venta sintética confirmada por el vendedor

20. Producción

Administración autoriza fabricación por producto/línea tras revisar necesidad y disponibilidad. Puede ajustar o rechazar y conservar motivo cuando corresponde. Producción real, consumos, calidad y cumplimiento dependen del sistema operativo conectado; en el repo se prueban principalmente escenarios locales simulados.

El límite clave es la separación de responsabilidades: una venta registrada expresa compromiso comercial; no crea por sí sola una orden de fabricación. Administración revisa cada línea, puede ajustar o rechazar y deja responsable/fecha/nota. Sólo tras autorización puede sincronizarse el vínculo con Odoo. README describe compra, fabricación, entrega y documentos como seguimiento de Odoo, manteniendo XLSX como alternativa que registra compartir archivo, no producción real.

21. Integración Odoo

`operations/odoo.py`, `operations/source_sync.py` y el addon `odoo_addons/vitali_lab` implementan mapeos y flujos de sincronización con referencias de venta, reservas, fabricación, entrega/devolución y documentos financieros en laboratorio. El conector técnico y tests están presentes; el README indica que el laboratorio es local y sintético, no una conexión productiva ni fiscal.

Conector `operations/odoo.py` intercambia referencias idempotentes y revisiones, maneja sincronización/snapshot/error y compara estados relevantes. El addon `vitali_lab` expone integración limitada en Odoo Community, sin parchear el núcleo. `scripts/preparar_odoo_lab.py` exige base `vitali_lab`, crea empresa/productos/lotes/stock/venta sintéticos y una producción

borrador; incluye aserción que confirma que la venta no crea fabricación automáticamente.

En el recorrido documentado, Odoo es autoridad de existencias compartidas, reservas, compra, fabricación, picking/entregas, devoluciones, factura, nota de crédito y cobro. SmartOrder conserva recomendación, cliente/ventas, revisión, planificación, autorización y seguimiento; así evita duplicar contabilidad, inventario físico y ERP. El README aclara que una prueba del conector no arrancó servidor HTTP: no prueba transmisión live ni ambiente fiscal.

El laboratorio usa empresa/códigos, receta, unidades y transacciones inventadas para verificar transiciones. Las cifras de pedido S00011 y saldos que describe README son un caso sintético reconciliado, no datos de Vitali ni evidencia tributaria salvadoreña.

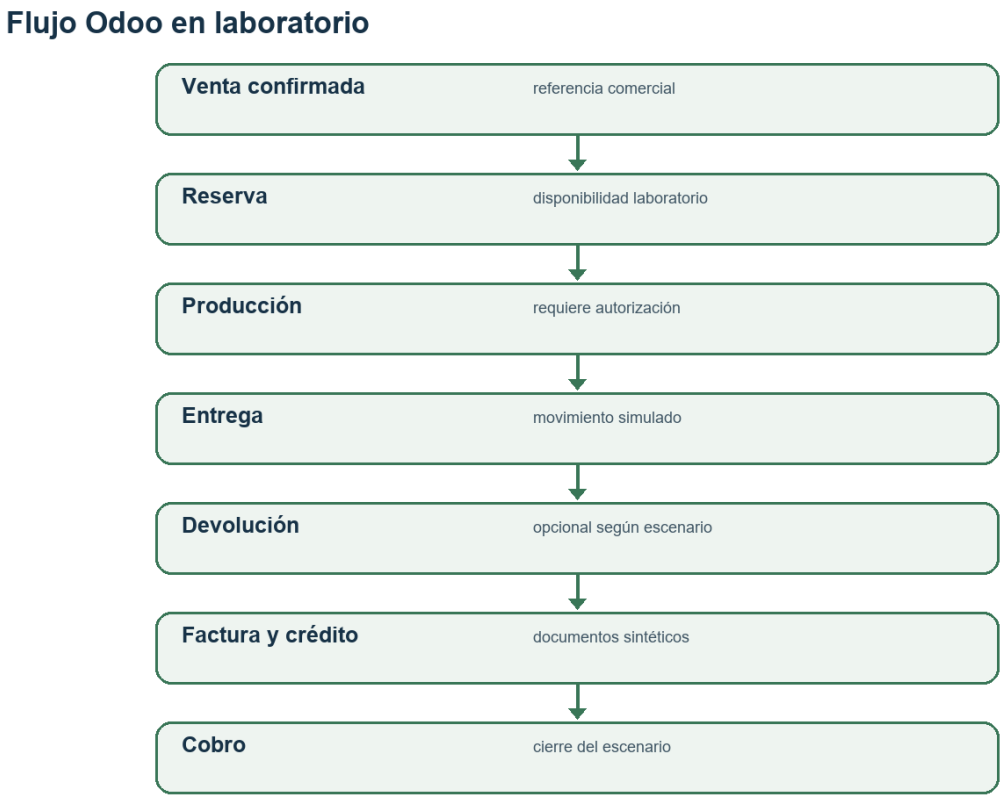


Figura 14. Flujo Odoo en laboratorio

22. Dashboards

Administración ve ventas y filtros, cargas, cobertura, inventario, pedidos pendientes, autorización, planificación, panel de piloto, sincronización y reportes. Vendedor ve cartera, disponibilidad, sugerencias, compromisos y seguimiento con límites de datos. Gráficos y KPIs proceden de las fuentes filtradas y snapshots. Métricas simuladas del laboratorio no describen actividad real.

Pantalla de conexión/catálogos Odoo en modo local. Administrador; mapeos del laboratorio. Permite revisar preparación y estado técnico, no acredita una conexión productiva.

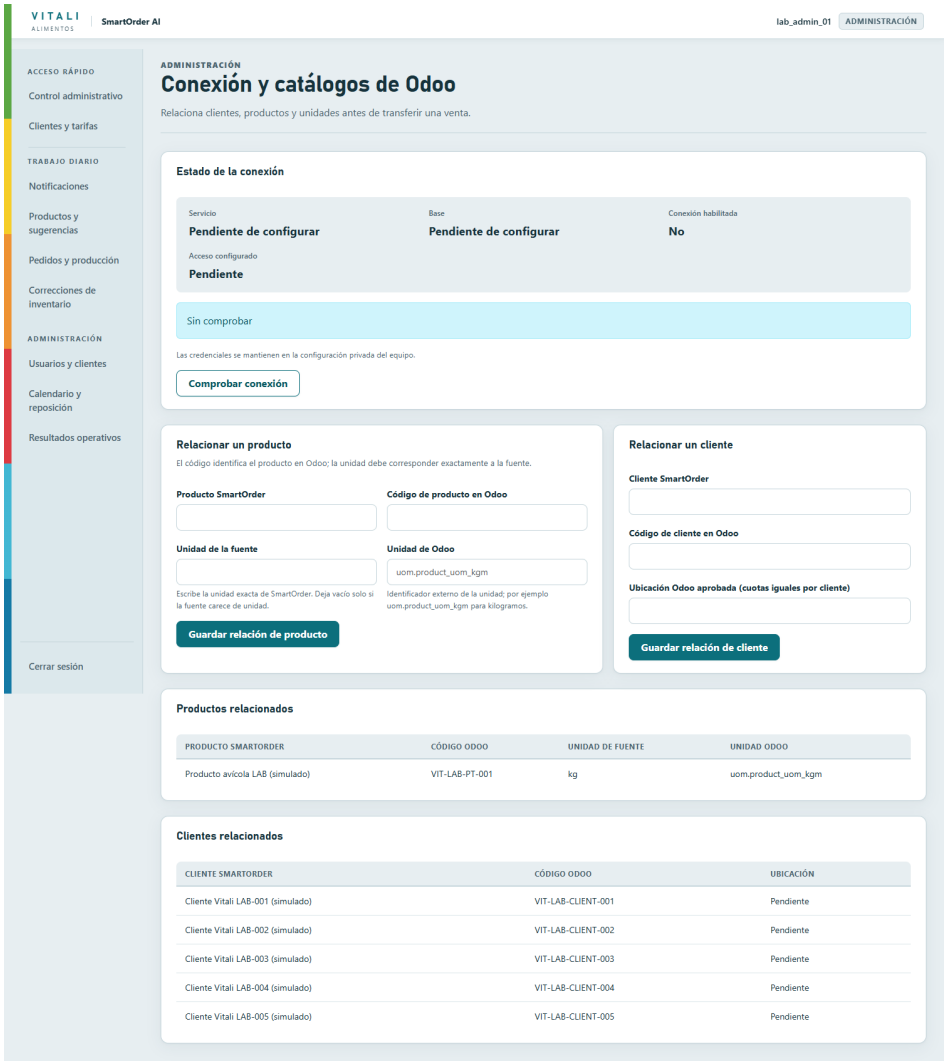


Figura 15. Pantalla de conexión/catálogos Odoo en modo local

23. Reportes

Hay informes operativos y vistas para ventas, existencia/movimientos, pedidos, fabricación, diferencias, sincronización y ciclo Odoo; los scripts también generan/exportan XLSX y artefactos del laboratorio. Un archivo exportado no prueba que producción lo haya recibido; el estado depende de confirmaciones/manualidad documentadas.

Reporte administrativo de resultados operativos. Dataset de laboratorio; ayuda a observar cantidades, entregas y estados, sin atribución a resultados comerciales reales.

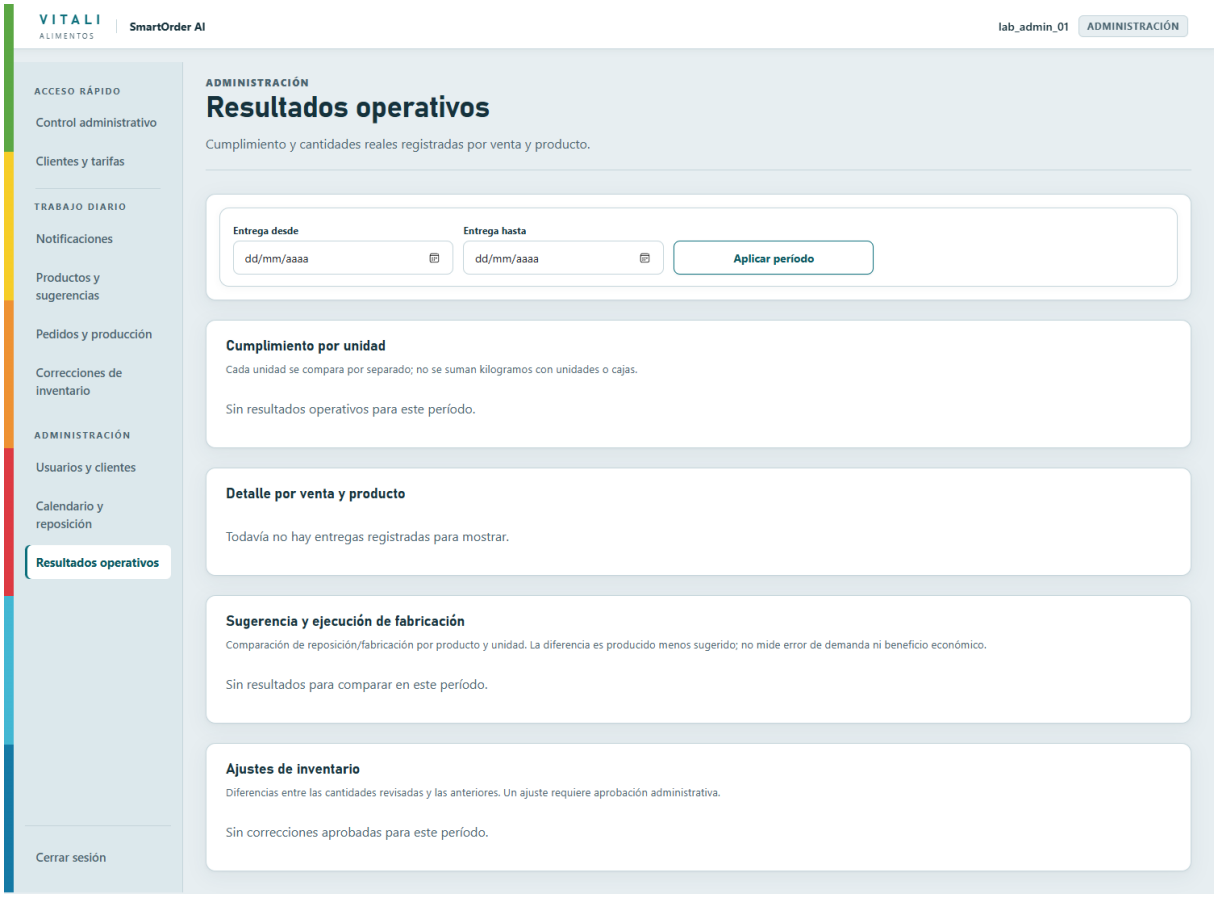


Figura 16. Reporte administrativo de resultados operativos

24. Streamlit

`streamlit_app.py` es la UI nativa; `operations/streamlit_bridge.py` remite acciones al mismo backend Django para compartir auth, permisos y servicios. Streamlit no debería reproducir reglas comerciales. `app.py` funciona como entrada de despliegue según README. Community Cloud puede suspender y reiniciar archivos locales.

El README sitúa app Cloud en Streamlit Community Cloud con entrypoint `app.py`, que delega a `streamlit_app.py`. En local se instala el mismo stack y se apuntan Django y Streamlit a `.local-web`; en la prueba del documento ambas sesiones operaron sobre `.local-web-lab` sintético. Auth bridge prepara settings y pasa acciones a servicios, con control de identidad/rol; la UI Streamlit utiliza controles propios en vez de templates Bootstrap.

Cloud requiere `SMARTORDER_SECRET_KEY` y `SMARTORDER_SETUP_CODE` en Secrets para primera cuenta según README. Archivos y DB locales pueden desaparecer al suspender/redeploy; no subir archivos, credenciales ni datos empresariales. Cambiar branch/entrypoint en Cloud debe verificarse porque el historial modificó enlace y app activa.

Pantalla de acceso de Streamlit conectada a la autenticación compartida Django. Captura local; no expone credenciales.

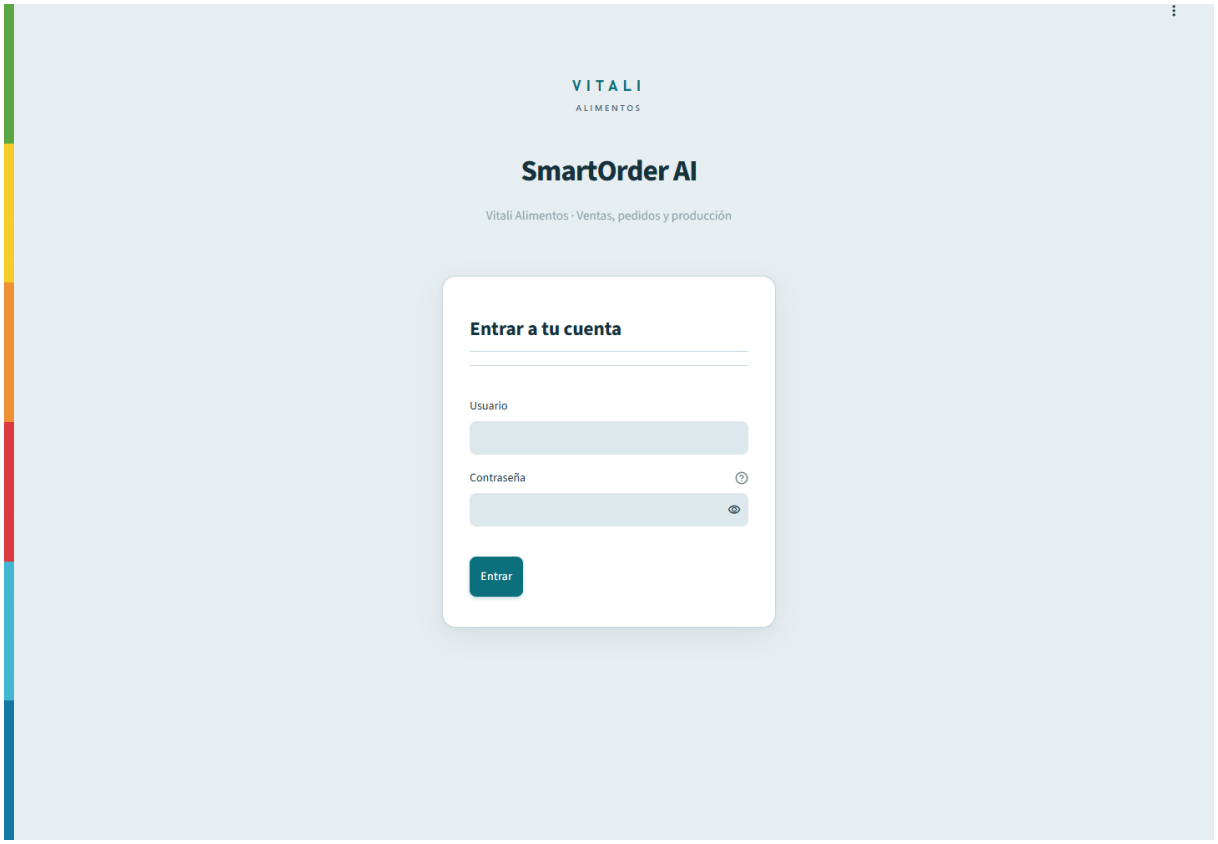


Figura 17. Pantalla de acceso de Streamlit conectada a la autenticación compartida Django

Panel administrativo de Streamlit con cifras sintéticas del laboratorio; permite revisar indicadores y carga de trabajo.

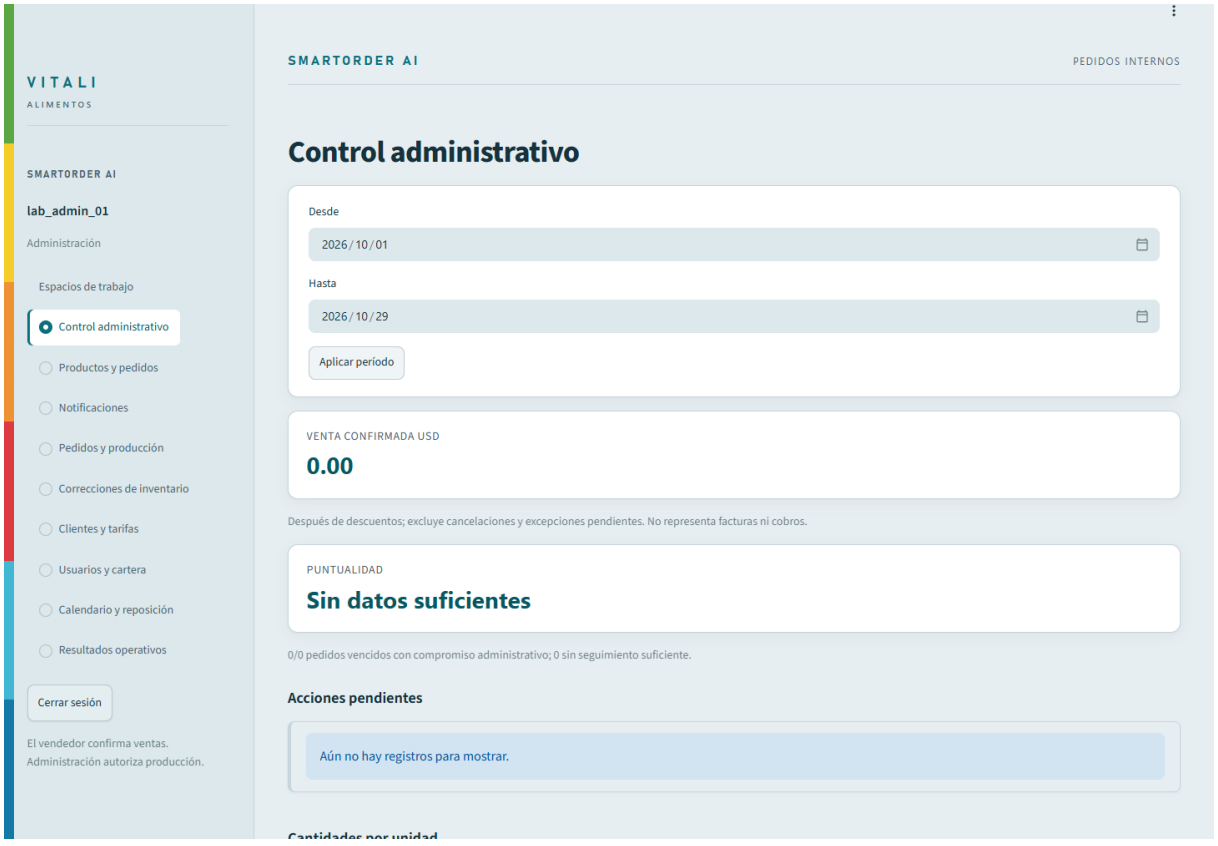


Figura 18. Panel administrativo de Streamlit con cifras sintéticas del laboratorio; permite revisar indicadores y carga de trabajo.

Panel de Streamlit del vendedor con cartera sintética; permite revisar sus sugerencias y seguimiento bajo permisos de su rol.

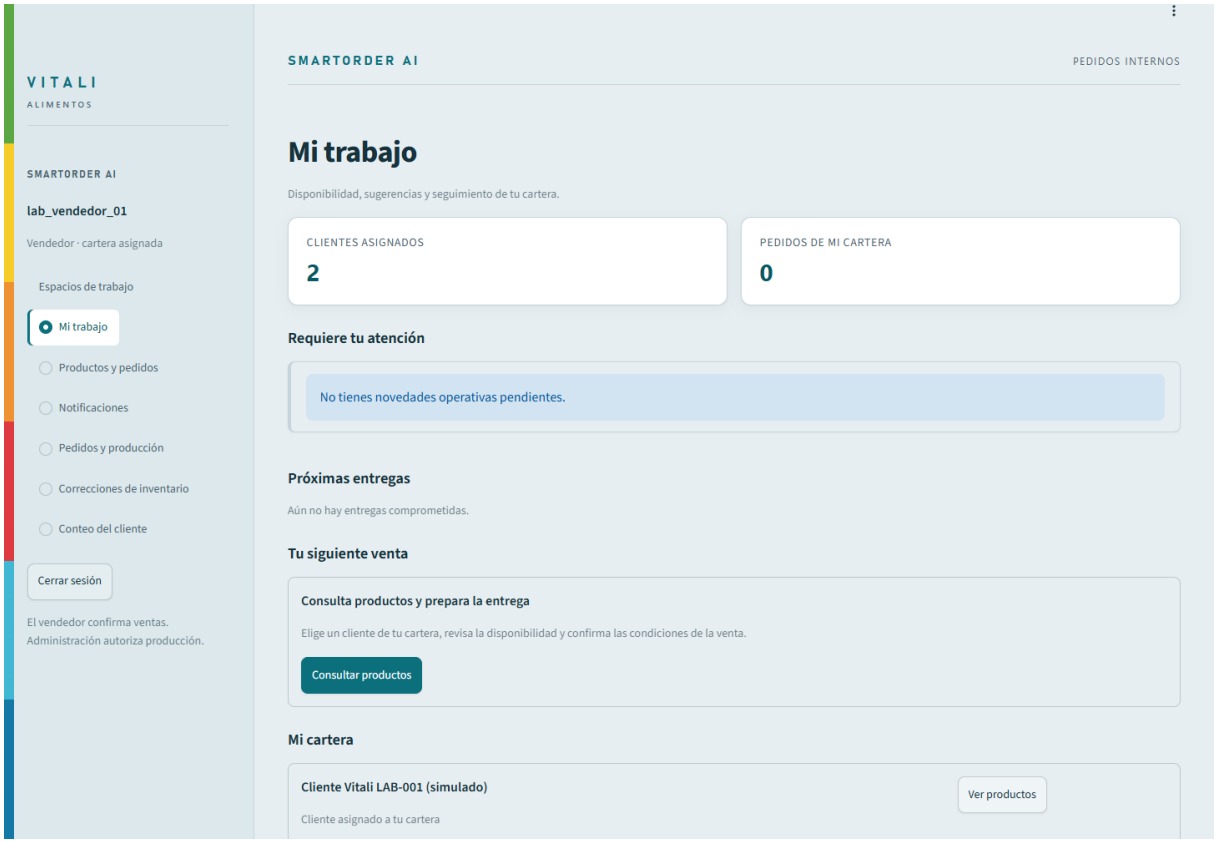


Figura 19. Panel de Streamlit del vendedor con cartera sintética; permite revisar sus sugerencias y seguimiento bajo permisos de su rol.

25. Django

Django ya no es una demo: administra sesión, usuarios, modelos, migraciones, permisos, ventas, inventario, pedidos, planificación, vistas e integración. Las reglas importantes residen en services/commercial y se reutilizan desde ambas interfaces.

`vitali_web/settings.py` selecciona DB local o `SMARTORDER_DATABASE_URL`, rutas WEB_HOME/media y seguridad de sesión. `operations/models.py` define restricciones e índices; `operations/migrations/` lleva evolución; `operations/views.py` maneja rutas web; `operations/services.py` concentra importación, reglas y operaciones; `operations/commercial.py` aplica criterios comerciales; `operations/odoo.py` integra ERP. Compatibilidad de hash legado permite migrar cuentas SQLite previas sin mutar la base antigua, mediante comando de importación de solo lectura.

26. Seguridad

Se utiliza autenticación Django, permisos por rol, validación de acciones, hashers incluyendo compatibilidad de hashes heredados, setup code remoto y secrets de entorno. El mínimo de password cambió a ocho caracteres en commit `9a533bc`. Secrets, claves, tokens Telegram y credenciales Odoo deben quedar fuera de Git. SQLite/archivos efímeros y exposición de demo son riesgos operativos.

Controles evidenciados incluyen autenticación Django, grupos/flags, decoradores y validaciones de servicio, CSRF en formularios, restricciones de base, hashing Django y LegacySmartOrderHasher, setup de uso único y secreto privado de arranque remoto. El commit `9a533bc` baja contraseña mínima a ocho caracteres, con impacto en política de contraseñas. La compatibilidad legado preserva login anterior, por eso cambio de hash/version se prueba.

Variables sensibles incluyen Django secret, setup code, Odoo admin/password y Telegram bot token/chat ID. Se mantienen en entorno o archivos locales ignorados; el material de documentación jamás reproduce valores. Los scripts del laboratorio limpian variables heredadas de Telegram antes de sembrar para impedir notificaciones externas. Aun con estos controles, demo pública y DB local no son frontera suficiente para producción multiusuario.

27. Pruebas y resultados técnicos comprobados

La auditoría reproduce resultados técnicos en el SHA actual: dependencias declaradas instaladas en entorno aislado; `manage.py check` correcto; `makemigrations --check --dry-run` sin cambios; 134 pruebas aprobadas; Streamlit arrancó y `/` y `/_stcore/health` devolvieron HTTP 200. La prueba manual capturó las dos interfaces y el flujo de una venta sintética en el laboratorio. Estos datos miden funcionamiento técnico del prototipo, no resultados comerciales. `pip check` y cobertura no se ejecutaron.

A. RESULTADOS TÉCNICOS COMPROBADOS. La instalación, arranque, suite y smoke tests se ejecutaron en el estado auditado: 134 pruebas aprobadas, Django check correcto, migraciones consistentes, Streamlit HTTP 200 y operación de pantallas con datos de laboratorio. La prueba de interfaz confirmó una venta sintética y su detalle, con producción pendiente de autorización.

Categorías derivadas de los módulos de prueba: `operations/tests.py` cubre ETL, snapshots, permisos, ventas, correcciones, alta/contraseña y revalidación; `test_sales_roles.py` roles/segregación; `test_commercial_pilot.py` métricas/fechas comerciales; `test_planning_inbox.py` planificación/duplicados/flujo; `test_inventory.py` inventario y unidades; `test_recommendations.py` candidatos y evaluación; `test_odoo_integration.py` conector/estados; `test_operational_ui.py` vistas y operación; `test_streamlit.py` bridge/navegación Streamlit; `test_brand_ui.py` estructura visual. No asigno recuentos por categoría sin extracción por clase/método.

El README conserva la batería histórica de 83 pruebas (ejecutada el 4 de octubre, 61.545 s); la ejecución reproducida en este SHA registró 134 tests en 122.344 s. Las bases de UI son temporales y parte de Odoo se simula/aisla. El run actual no ejecutó cobertura ni `pip check`; tampoco prueba el reinicio efímero de Cloud.

Evidencia de ejecución reproducida

Referencia	origin/main · 5129401fd8e346f088ba377aff039d44eb03a1d3
Dependencias	Instaladas en entorno virtual aislado
Suite	134 pruebas aprobadas · 122.344 s
Django	manage.py check correcto
Migraciones	makemigrations --check --dry-run sin cambios
Streamlit	/ y /_stcore/health respondieron HTTP 200
No ejecutado	pip check y cobertura
No comprobado	Persistencia Cloud y Odoo HTTP productivo

28. Resultados de negocio esperados

Los efectos previstos son mejorar consistencia y seguimiento comercial, visibilidad de compromisos de producción, disponibilidad por fecha, trazabilidad y apoyo a decisiones de reposición. Cifras de 3–6.5% de recuperación de ventas o 20–40% menos desperdicio son referencias potenciales sujetas a piloto y medición; no son resultados demostrados por esta auditoría.

B. RESULTADOS DE NEGOCIO ESPERADOS. Menos errores manuales, mejor disponibilidad, planificación y seguimiento son hipótesis que deben medirse con un piloto comparativo. Las referencias potenciales de 3–6.5% y 20–40% no son metas logradas ni resultados medidos con datos sintéticos.

29. Limitaciones

Calidad y cobertura del Excel limitan el forecast; datos sintéticos no validan comportamiento comercial; unidades/SKU/ubicaciones deben mapearse; Streamlit Cloud no persiste archivos localmente; Odoo es laboratorio; no se acreditó integración real, respaldo automatizado, despliegue escalable ni CI de GitHub. El horizonte y granularidad no equivalen a planificación completa para más de 100 salas/tiendas.

Fuentes son agregados históricos cargados y snapshot; la ausencia de tienda/sucursal/SKU en el Excel limita granularidad, salvo que se incorpore y valide catálogo/mapeo. La semilla lab crea un producto y asignaciones para representar flujo, no variedad ni estacionalidad del negocio. En Cloud SQLite/medios no garantizan retención. Odoo local y addon no equivalen a red productiva, permisos definitivos, DTE/fiscalidad o conciliación real. README expresamente presenta cifras del laboratorio y su caso contable como sintéticos.

No se midió entrenamiento/latencia/uso concurrente bajo carga, no existe evidencia de despliegue con PostgreSQL ni restore de backup en esta auditoría, y no hay workflows CI versionados observados. Por tanto, 134 tests es evidencia de comportamiento de código aislado, no certificado de disponibilidad, escalabilidad o continuidad de negocio.

30. Deuda técnica

`operations/views.py` y `operations/services.py` son extensos; mantener límites de dominio será relevante. Hay numerosos scripts de escenarios que requieren documentación/automatización. Configuración Django/Streamlit y filesystem demanda disciplina. Falta pipeline CI observado y medición de cobertura en la auditoría. Debe diferenciarse sandbox local de operación multiusuario.

Puntos que conviene abordar después: dividir `operations/views.py` y `services.py` manteniendo transacciones, límites de permiso e idempotencia; mantener API compartida para Django/Streamlit; automatizar escenarios repetibles desde scripts; documentar cuál catálogo es autoridad; fijar dependencias/ambientes de Cloud; agregar CI, cobertura y smoke de despliegue; definir backups y restore de DB/media; agregar pruebas con Odoo HTTP en sandbox protegido.

La duplicación no debe inferirse solo por tamaño; hay pruebas de fronteras en `tests/test_boundaries.py`. La prioridad es preservar consistencia de revalidación, cobertura, unidades y audit trail antes de refactorizar.

31. Estado actual vs. visión objetivo

Aspecto	Visión	Estado actual	Brecha
Más de 100 salas/tiendas	escala corporativa	prototipo y pocos puntos sintéticos	carga, permisos y pruebas de escala
Tienda/SKU	granularidad fina	depende de mapeos y datos disponibles	catálogo maestro real
ERP	operación integrada	laboratorio local Odoo	endpoint/credenciales y pruebas productivas
IA	demanda fiable	método candidato con backtest	historia representativa y seguimiento de deriva
Producción	plan operativo	autorización y escenarios	conexión a planta/MRP
Inventario	saldo actualizado	carga/snapshot y movimientos	fuente de verdad sincronizada
Despliegue	servicio estable	demo Streamlit Cloud	entorno y monitoreo
Persistencia	multiusuario durable	SQLite/local por defecto	PostgreSQL y almacenamiento/backup

32. Próximos pasos

Para una entrega académica: presentar el flujo con datos sintéticos claramente rotulados y explicar límites de Cloud/Odoo. Para piloto: aprobar arquitectura de persistencia y backups, catálogo y unidades oficiales, seguridad de cuentas, conexión ERP controlada, monitoreo, pruebas de aceptación y evaluación del forecast en datos de varias temporadas. No cargar datos reales en la demo Cloud.

33. Conclusiones

SmartOrder AI evolucionó a un prototipo de operación comercial con dos UI y dominio compartido. La recomendación es supervisada; la aprobación de producción es separada; Odoo y volúmenes demostrativos son sintéticos. La evidencia técnica actual es sólida para prototipo (134 tests y smoke de Streamlit), pero no demuestra durabilidad en Cloud, integración ERP productiva ni impacto económico.